

Relatório Técnico

Simulador Didático de Sistemas Operacionais
Simulação orientada a eventos discretos — processos e threads, escalonamento de CPU, memória paginada, E/S e sistema de arquivos

Disciplina de Sistemas Operacionais
Prof. Maurício Acconcia Dias

Implementação em C# / .NET 8

1. Introdução e objetivo

Este trabalho implementa um simulador didático que reproduz, em nível de modelo, o comportamento dos principais subsistemas de um sistema operacional. O objetivo não é obter desempenho real, mas **observar e medir** o efeito de diferentes políticas — de escalonamento de CPU, de substituição de páginas e de escalonamento de disco — sobre métricas clássicas como tempo de retorno, tempo de espera, utilização da CPU e taxa de faltas de página.

Não há paralelismo real: processos, threads, dispositivos e o próprio processador são **objetos de modelo**. O tempo avança por um **relógio lógico**, em unidades abstratas (u.t.), controlado por uma fila de eventos futuros. Toda a lógica de domínio está na biblioteca `SimuladorSO.Nucleo`; a aplicação de linha de comando `SimuladorSO.Cli` apenas lê parâmetros, monta a simulação e imprime o relatório.

2. Arquitetura geral

O coração do simulador é uma **fila de eventos** (min-heap) ordenada por (instante, sequência). O campo de sequência é um contador monotônico que garante **desempate determinístico** entre eventos agendados para o mesmo instante: a mesma carga, com a mesma configuração e a mesma semente, produz sempre o mesmo log e as mesmas métricas. Essa propriedade é verificada por teste automatizado.

Os tipos de evento são: chegada de processo, despacho, execução de uma unidade de CPU, fim de atendimento de falta de página e conclusão de E/S. O núcleo consome o evento de menor tempo, avança o relógio até ele e o executa; a execução pode agendar novos eventos. O laço termina quando a fila esvazia.

Separação mecanismo × política. O núcleo (mecanismo) não conhece as políticas concretas. Ele depende apenas de três interfaces, cujas implementações são injetadas pela `FabricaSimulador`:

- `IEscalonador` — FCFS, Round Robin e Prioridade;
- `IPoliticaSubstituicao` — FIFO e LRU;
- `IEscalonadorDisco` — FCFS, SSTF e SCAN.

Trocar de política é trocar a implementação injetada, sem alterar o núcleo — o que mantém baixo o acoplamento e facilita a extensão.

3. Processos e threads

A **thread** é a unidade escalonável. Cada thread carrega um fluxo sequencial de operações (surto de CPU, operações de E/S e operações de arquivo) e possui um TCB (contador de programa lógico, registradores e pilha lógica). O processo agrupa threads e mantém o PCB (prioridade, espaço de endereçamento — a tabela de páginas — e a tabela de arquivos abertos). Um processo termina quando **todas** as suas threads terminam.

Os estados da thread e todas as transições são registrados no log:

Transição	Evento que a provoca
Nova → Pronta	Admissão na chegada do processo

Transição	Evento que a provoca
Pronta → Executando	Despacho pelo escalonador
Executando → Pronta	Preempção (fim de quantum ou prioridade maior)
Executando → Bloqueada	E/S bloqueante ou falta de página
Bloqueada → Pronta	Interrupção de conclusão de E/S / fim do atendimento da falta
Executando → Finalizada	Fim da última operação da thread

Tabela 1 — Transições de estado das threads.

4. Escalonamento de CPU

A CPU executa em **passos unitários**: cada unidade de tempo consumida por um surto é um evento. Essa granularidade permite que conclusões concorrentes (por exemplo, o fim do atendimento de uma falta de página de outra thread) sejam ordenadas corretamente e que o quantum e a preempção sejam avaliados em fronteiras bem definidas.

Troca de contexto. A cada despacho é cobrado um custo configurável (--custo-troca), durante o qual a CPU fica ocupada. Esse tempo é contabilizado como **sobrecarga**, separado da fração útil; o número de trocas também é medido. A ociosidade é o que sobra: ocioso = total – útil – sobrecarga.

Políticas. FCFS é não preemptivo. Round Robin usa quantum configurável e preempta ao esgotá-lo. A política de Prioridade admite os modos preemptivo e não preemptivo (declarados na linha de comando) e possui *aging* opcional como extensão. Menor número indica maior prioridade. O **desempate** é sempre determinístico: prioridade efetiva, depois instante de chegada e, por fim, a ordem de identificação da thread.

5. Gerência de memória

Cada processo tem um espaço de endereçamento lógico próprio e uma **tabela de páginas** (bit de presença e moldura). Um endereço lógico é traduzido em (página = endereço / tamanho_da_página, deslocamento = endereço mod tamanho). No acesso, se a página está presente conta-se um **acerto**; caso contrário, uma **falta**: a thread **bloqueia** durante o tempo de atendimento e, ao final, a página é carregada. Havendo moldura livre, ela é usada; caso contrário, a política de substituição escolhe a vítima. FIFO (mínimo obrigatório) e LRU estão implementadas. Não há espera ocupada: o desbloqueio ocorre por evento.

6. Entrada e saída

Há dispositivos de **bloco** (disco) e de **caractere** (console). Cada dispositivo tem fila própria, estado (livre/ocupado) e tempo de serviço configurável. Numa operação **bloqueante**, a thread suspende até a **interrupção** de conclusão devolvê-la à fila de prontos; a versão **não bloqueante** não suspende a thread. Não há espera ocupada em nenhum caso.

O disco suporta três políticas de escalonamento: FCFS (mínimo), SSTF e SCAN (elevador). O simulador mede o **deslocamento total do cabeçote**, que serve de proxy para o custo mecânico de busca e é a métrica comparada no Experimento 3.

7. Sistema de arquivos

O sistema de arquivos é **hierárquico** a partir da raiz. Cada nó (arquivo ou diretório) tem um descritor com nome, tipo, tamanho, permissões (leitura/escrita), timestamps lógicos e a localização dos blocos. As operações suportadas são criar arquivo/diretório, abrir, ler, escrever, fechar, remover e listar. Ao abrir, cria-se um **descritor por processo** ligado a uma tabela global de arquivos abertos; fechar libera o descritor.

Operações inválidas são detectadas e registradas como erro, sem corromper o estado: ler por um descritor fechado, remover um caminho inexistente, remover um arquivo aberto, recriar um nó existente e violações de permissão.

Alocação encadeada. Os blocos de um arquivo formam uma lista ligada. A vantagem é não sofrer fragmentação externa e crescer sem realocação. A **limitação documentada** é o acesso aleatório ineficiente: alcançar o bloco k exige percorrer a cadeia desde o início, tornando o acesso essencialmente sequencial.

8. Métricas

Todas as métricas derivam do relógio lógico. As fórmulas usadas são:

- **Tempo de retorno** (turnaround) = instante de término – instante de chegada;
- **Tempo de espera** = tempo acumulado na fila de prontos;
- **Tempo de resposta** = instante da primeira ida à CPU – chegada;
- **Utilização da CPU** = unidades úteis / tempo total (a sobrecarga de troca é reportada à parte);
- **Vazão** (throughput) = processos concluídos / tempo total;
- **Utilização de dispositivo** = tempo ocupado / tempo total;
- **Taxa de faltas** = faltas / total de referências;
- **Trocas de contexto** = contagem + sobrecarga total acumulada.

9. Experimentos

Todos os experimentos são reproduzíveis por `bash experimentos/rodar_experimentos.sh`. O custo de troca de contexto foi mantido no padrão (1 u.t.) e a semente fixada, garantindo resultados idênticos entre execuções.

9.1 Experimento 1 — FCFS × Round Robin × Prioridade (carga mista)

Mesma carga (`cargas/mista.txt`, quatro threads em três processos, com CPU, E/S e arquivos) submetida às três políticas. Round Robin com quantum 4; Prioridade nos modos preemptivo (P) e não preemptivo (NP).

Política	T. total	Retorno méd.	Espera méd.	Resposta méd.	Util. CPU	Trocas
FCFS	78	55,00	12,75	1,25	41,03%	27
Round Robin q=4	78	56,25	13,50	1,25	41,03%	28
Prioridade (P)	92	56,75	14,75	1,25	34,78%	28
Prioridade (NP)	84	54,75	12,75	1,25	38,10%	27

Tabela 2 — Comparação de escalonadores de CPU sobre a carga mista.

Interpretação. Como a carga tem muita E/S, as threads cedem a CPU frequentemente por conta própria, e FCFS e Round Robin ficam quase empatados — o quantum 4 raramente é atingido, gerando só uma preempção a mais. A Prioridade preemptiva apresenta o pior tempo total e a menor utilização: as preempções por chegada de thread mais prioritária aumentam o número de trocas e, com isso, a sobrecarga, esticando o tempo total para 92 u.t. O modo não preemptivo recupera parte disso. A lição é clara: preempção melhora a responsividade a processos prioritários, mas cobra o preço da troca de contexto.

9.2 Experimento 2 — variação do quantum no Round Robin (carga mista)

A mesma carga mista sob Round Robin, variando o quantum de 1 a 16 u.t.

Quantum	T. total	Retorno méd.	Espera méd.	Util. CPU	Trocas
1	101	75,75	33,75	31,68%	52
2	79	57,25	15,00	40,51%	30
4	78	56,25	13,50	41,03%	28
8	78	55,00	12,75	41,03%	27
16	78	55,00	12,75	41,03%	27

Tabela 3 — Efeito do quantum sobre o Round Robin.

Interpretação. Um quantum muito pequeno (1) é desastroso: são 52 trocas de contexto, a utilização da CPU cai para 31,68% e o tempo total sobe para 101 u.t., pois a sobrecarga de troca domina. À medida que o quantum cresce, o número de trocas diminui e o desempenho melhora; a partir de um quantum grande o comportamento **converge para o FCFS** (quantum 8 e 16 já são idênticos entre si e iguais ao FCFS da Tabela 2), porque as fatias passam a ser maiores do que os surtos de CPU. O quantum é, portanto, um compromisso entre responsividade e sobrecarga.

9.3 Experimento 3 — memória (FIFO × LRU) e disco (FCFS × SSTF × SCAN)

Parte A — substituição de páginas. Carga `cargas/padrao_memoria.txt`, com tamanho de página 1 e um conjunto de trabalho fixo {1, 2} reutilizado a cada ciclo, mais uma terceira página que sempre muda. Comparam-se FIFO e LRU com 3 e 4 molduras.

Política	Molduras	Faltas	Taxa de faltas	T. total	Util. CPU
FIFO	3	11	64,71%	95	17,89%
LRU	3	7	41,18%	67	25,37%
FIFO	4	9	52,94%	81	20,99%
LRU	4	7	41,18%	67	25,37%

Tabela 4 — FIFO × LRU sobre um padrão com reutilização.

Interpretação. Com 3 molduras, LRU sofre 7 faltas contra 11 do FIFO — uma redução de ~36%. Como a thread faltante bloqueia durante o atendimento, menos faltas significam menos tempo bloqueado: o tempo total cai de 95 para 67 u.t. e a utilização da CPU sobe. O motivo é que LRU preserva as páginas 1 e 2 (sempre as mais recentemente usadas) e só troca a moldura rotativa, ao passo que FIFO, ignorando a reutilização, expulsa 1 e 2 periodicamente. Vale notar que o resultado depende do padrão de acesso: em sequências sem reutilização as duas políticas podem empatar.

Parte B — escalonamento de disco. Carga `cargas/io_disco.txt`: cinco requisições chegam à fila do disco praticamente juntas (posições 20, 190, 50, 160, 90; cabeçote inicia em 0), permitindo o reordenamento.

Política	T. total	Requisições	Desloc. do cabeçote
FCFS	43	5	510
SSTF	43	5	190
SCAN	43	5	190

Tabela 5 — FCFS × SSTF × SCAN: deslocamento do cabeçote.

Interpretação. FCFS atende na ordem de chegada e percorre 510 unidades; SSTF e SCAN, ao reordenarem para reduzir o movimento, percorrem apenas 190 — uma redução de ~63%. O tempo total permanece 43 u.t. nas três porque, neste modelo abstrato, o tempo de serviço é fixo e independente da distância, e o disco (totalmente ocupado) é o gargalo. Num modelo em que o tempo de serviço fosse proporcional ao deslocamento, essa economia de movimento se traduziria também em menor tempo total.

10. Testes automatizados

A suíte é executada com `dotnet run --project tests/SimuladorS0.Testes -c Release` e usa um arcabouço de asserções próprio, sem dependências externas (roda offline). São 63 verificações, todas passando, cobrindo memória (tradução de endereços, falta seguida de acerto, vítimas FIFO e LRU), escalonamento (ordem FCFS, fim de quantum, seleção e preempção por prioridade), sistema de arquivos (ciclo completo, descritores e posição, operações inválidas, permissões, diretórios) e integração (métricas conferidas à mão, transições de estado com E/S, contagem de preempções e determinismo). O processo retorna código 0 em sucesso e 1 em falha, adequado a CI.

11. Limitações

- **Alocação encadeada** no sistema de arquivos: acesso aleatório exige percorrer a cadeia, ficando essencialmente sequencial.
- **Modelo de tempo abstrato**: o tempo de serviço do disco não depende do deslocamento do cabeçote, o que limita o impacto de SSTF/SCAN no tempo total (embora o deslocamento seja corretamente medido).
- **Passo unitário de CPU**: preempções e verificações ocorrem em fronteiras de unidade, introduzindo latência de no máximo uma unidade — coerente com a granularidade do relógio lógico.
- **Custo de troca a cada despacho**: a sobrecarga é cobrada em todo despacho, inclusive no retorno de bloqueio, o que é uma simplificação conservadora.
- **Sem paralelismo real**: há uma única CPU; múltiplos núcleos não são modelados.

12. Conclusão

O simulador cumpre os requisitos do enunciado: modela processos e threads com transições de estado registradas, três políticas de escalonamento de CPU com troca de contexto contabilizada, memória paginada com FIFO e LRU, E/S bloqueante e não bloqueante com dispositivos de bloco e caractere e três políticas de disco, além de um sistema de arquivos hierárquico com detecção de operações inválidas. Os experimentos confirmam efeitos teóricos conhecidos — o compromisso do quantum, o custo da preempção, a vantagem do LRU sob reutilização e a economia de movimento do SSTF/SCAN — todos medidos a partir do relógio lógico. A arquitetura orientada a objetos, com clara separação entre mecanismo e política, torna o sistema fácil de estender e de testar, como evidenciam as 63 verificações automatizadas e a reprodutibilidade determinística das execuções.